Cyclic Redundancy Check (CRC) & Attendance Auditor

Comprehensive Slide-by-Slide Explanation & Presentation Guide

Target Presentation: 8-Slide CRC Dashboard Summary	**Format:** Microsoft PowerPoint (.pptx)
Subject: Computer Networks / Data Integrity	**Implementation:** 100% Pure Python XOR Engine

1. Executive Overview & Website Purpose

This guide accompanies the 8-slide summary presentation explaining the interactive **CRC Verification & Attendance Integrity Dashboard**. The project demonstrates how Cyclic Redundancy Check (CRC)—the mathematical foundation behind modern networking protocols, Ethernet, and zip files—can be applied to solve real-world data auditing challenges such as verifying attendance records without any third-party black-box libraries.

Core Design Rules Implemented:

- **Pure Bitwise XOR Engine:** Uses only the user-specified crc(data, gen) and text_to_binary(text) functions.
- **Strictly No Flipping:** Excludes artificial bit-flipping controls; evaluates authentic user inputs and genuine altered files.
- **3 Comprehensive Modes:** Binary Codeword Verification, Text CRC Comparison, and File Attendance CSV Auditing.

2. Slide-by-Slide Detailed Explanation

Slide 1: Title & System Overview

Objective: Establishes the project scope and presents the three fundamental pillars.

Slide 1 introduces the audience to the CRC Integrity Dashboard. It highlights that the system is not just an abstract algorithm demonstration, but a functional audit tool designed to protect educational and institutional files from accidental modifications. Emphasize that it is powered by pure bitwise XOR polynomial division.

Key Presentation Points:

Key Talking Point: 'Today we present a complete Cyclic Redundancy Check system that bridges theoretical polynomial division with real-world file integrity verification.'

Badges to Highlight: Pure Python XOR Engine, Zero Bit-Flipping, and Attendance CSV File Auditing.

Slide 2: Problem Statement — The Attendance Dilemma

Objective: Presents the real-world scenario from the classroom case study slide.

An instructor completes class and exports an attendance CSV file. Later, during storage on disk, transmission over network drives, or handling by assistants, rows can be deleted or values flipped (e.g., 'Absent' to 'Present'). Manual line-by-line verification is impossible for large classes. CRC solves this by generating a mathematically unique checksum fingerprint.

Key Presentation Points:

- **The Problem:** How can an instructor verify if a file has remained 100% authentic without manual checking?
- **The Solution:** Generate a reference CRC immediately after class. Later, recalculate the audited CRC. A match guarantees 100% authenticity; any mismatch instantly raises a tampering alert.

Slide 3: Theoretical Foundation — What is CRC & How Does it Work?

Objective: Breaks down modulo-2 binary arithmetic and the division formula in simple terms.

CRC is an error-detecting code based on polynomial division. Unlike regular long division, CRC uses **Modulo-2 arithmetic**, where addition and subtraction are identical to bitwise XOR (^). There are no carries or borrows. The sender pads data with k zeros, divides by generator G, and attaches the remainder R. The receiver divides the codeword by G; if the remainder is zero, the transmission had no errors.

Key Presentation Points:

- **XOR Rules:** 1 ^ 1 = 0, 0 ^ 0 = 0, 1 ^ 0 = 1, 0 ^ 1 = 1.
- **Mathematical Identity:** Data D(x) · 2^k + R(x) = Codeword C(x).
- **The Golden Rule:** If received Codeword divided by G produces Remainder = 0, no error occurred!

Slide 4: System Configuration — CRC Standards & Polynomials

Objective: Details the four built-in CRC standards and polynomial specifications.

The dashboard allows the user to choose from four standard CRC polynomial types or enter any custom binary divisor. The degree of the polynomial determines how many check bits are appended (CRC length = len(gen) - 1).

Key Presentation Points:

- **CRC-3:** Generator '1011' ($x^3 + x + 1$) — 3 check bits. Great for small-scale educational tracing.
- **CRC-4:** Generator '10011' ($x^4 + x + 1$) — 4 check bits. Used in telecommunication framing (ITU G.704).
- **CRC-8:** Generator '100000111' ($x^8 + x^2 + x + 1$) — 8 check bits (1 byte). ATM & industrial sensors.
- **CRC-32:** IEEE 802.3 33-bit binary generator — 32 check bits (4 bytes). Ethernet, ZIP, and PNG standard.

Slide 5: Mode 1 — Binary Input (Codeword Method)

Objective: Explains transmission verification through the classic codeword mechanism.

Mode 1 demonstrates the fundamental networking transmission model. The sender inputs a binary string, pads it with zeros, computes the CRC, and produces the complete transmitted codeword. The receiver receives the codeword and performs modulo-2 division. If the remainder is 0, the receiver extracts the original data with complete confidence.

Key Presentation Points:

- **Sender Step:** Padded = data + '0'*crc_len -> sender_crc = crc(Padded, gen) -> Codeword = data + sender_crc.
- **Receiver Step:** remainder = crc(received_codeword, gen).
- **Verdict Logic:** If int(remainder, 2) == 0 -> No Error Detected (Data extracted). Else -> Error Detected.

Slide 6: Mode 2 — Text Input (CRC Value Comparison)

Objective: Demonstrates converting natural language into binary streams and comparing checksums.

Mode 2 allows entering text messages. The system converts every character into standard 8-bit ASCII binary using text_to_binary(text). The sender computes its CRC, and the receiver computes its CRC. If both checksums match, the text is certified authentic. Even a single changed letter or case change completely shifts the CRC.

Key Presentation Points:

- **Binary Conversion:** ''.join(format(ord(char), '08b') for char in text).
- **Sensitivity Demonstration:** 'Computer Networks CRC Lab' yields CRC 00001000. Changing the capital 'L' to lowercase 'l' yields CRC 01001011 (complete mismatch!).

Slide 7: Mode 3 — File Input & Attendance CSV Integrity Audit

Objective: Solves the instructor's case study directly with automated audit reports.

Mode 3 is the core practical highlight of the dashboard. The instructor uploads or selects the pristine attendance CSV file. The auditor uploads the file under test. The dashboard parses the files, computes the respective CRC-8 / CRC-32 checksums, and compares them side-by-side. The dashboard provides an official downloadable CSV verification report.

Key Presentation Points:

- **Test 1 (Authentic):** Original CSV vs itself -> CRC 01111001 == 01111001 -> PASS.
- **Test 2 (Tampered Status):** Student 103 status changed from 'Absent' to 'Present' -> CRC 11101000 -> FAIL.
- **Test 3 (Deleted Row):** Student 105 row deleted -> CRC 10011110 -> FAIL.
- **Audit Report:** Downloadable CSV file with timestamp, filenames, CRCs, and pass/fail status.

Slide 8: Architecture Summary & Key Takeaways

Objective: Summarizes strict constraints, architectural choices, and technical takeaways.

Slide 8 reviews the technical rigor of the implementation: 100% adherence to the user's provided XOR algorithm without external libraries, elimination of artificial bit-flipping shortcuts, and automated unit testing verification via test_crc_rules.py.

Key Presentation Points:

- **Zero Cheating:** Built completely on first principles without zlib or hashlib.
- **Reliable Detection:** Mathematical proof that single-bit and burst errors cannot pass undetected.
- **Practical Utility:** An accessible web dashboard that empowers instructors to audit file records instantly.

3. Frequently Asked Questions (Viva Voce / Defense Guide)

Below are the most common questions an instructor or evaluator may ask during a project defense, accompanied by concise, technically accurate answers.

Q1: Why does CRC use Modulo-2 arithmetic instead of standard arithmetic?

Modulo-2 arithmetic operates without carry or borrow propagation. Every bit position is independent and evaluated purely through bitwise XOR (^). This makes modulo-2 division exceptionally fast, lightweight, and hardware-friendly while retaining rigorous error detection capabilities.

Q2: Why does a single modified character in Mode 2 or 3 produce a completely different CRC?

Because CRC treats data as a continuous polynomial stream. Changing a single character alters 1 or more bits in the binary dividend. During continuous long division, that bit change propagates through the XOR stages, leaving a completely different remainder polynomial.

Q3: What is the difference between Mode 1 (Codeword) and Modes 2 & 3 (Comparison)?

In Mode 1 (Codeword Verification), the CRC is physically appended to the data stream and sent across a channel; the receiver checks if dividing the entire packet yields zero remainder. In Modes 2 and 3 (Comparison), the sender and receiver compute standalone checksums independently and compare them as equality hashes.

Q4: Why was bit-flipping omitted in this dashboard?

Bit-flipping buttons create artificial, synthetic errors. In real-world audits (such as the attendance CSV case study), file corruption happens through human tampering, truncated rows, or file transfers. The dashboard tests authentic user input and actual modified files directly without shortcuts.

Q5: How does CRC detect row deletion in CSV files?

When a row is deleted, multiple newline ('\n') and student character bytes are removed. This shortens the binary bitstream by hundreds of bits, altering the dividend polynomial completely and causing an inescapable CRC mismatch.